# ROT

# Reddit Options Trader

ML Engine & 9-Stage Signal Pipeline — Technical Specification

**Classification: Confidential**

Intended Audience: Quantitative Analysts, Applied Mathematicians, ML Engineers

Version: 1.0 | February 2026

Author: Matthew Charles Busel — AI Engineer & Tech Writer

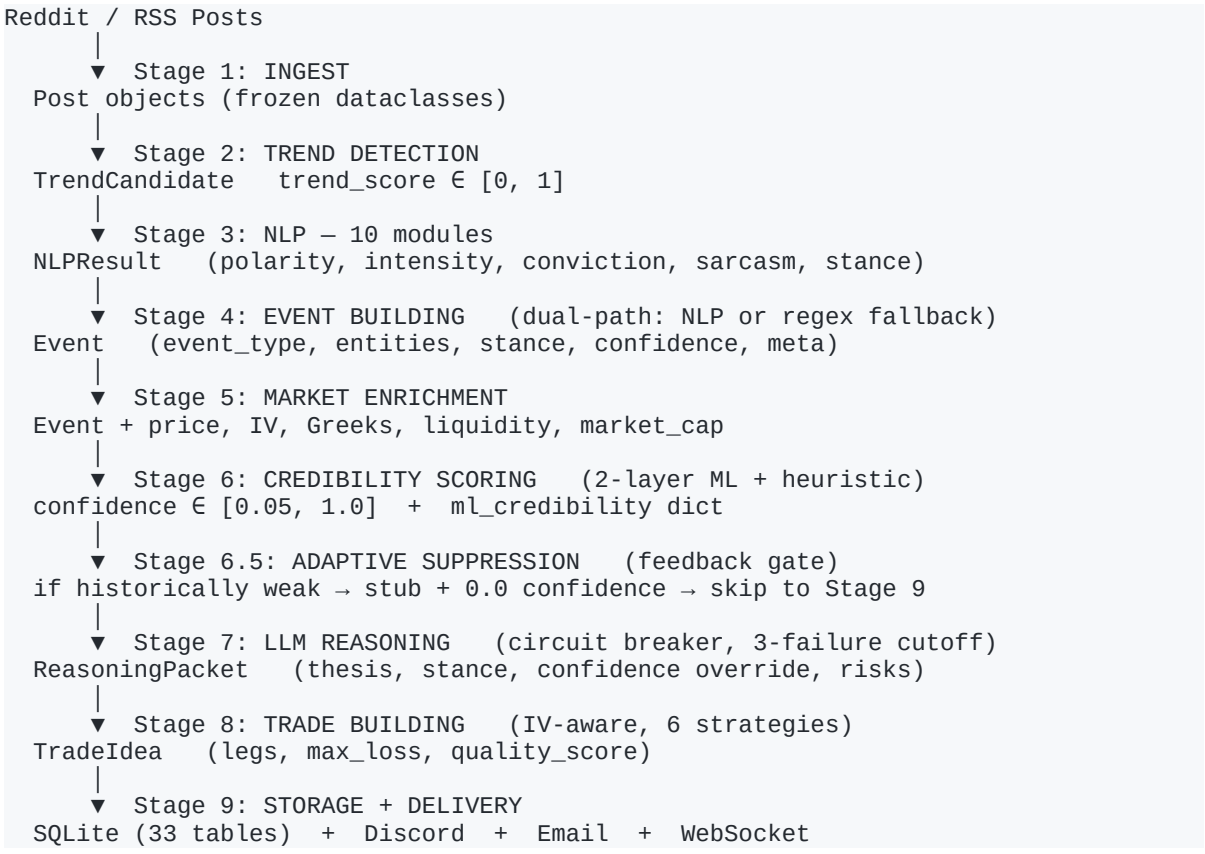
# Abstract

ROT (Reddit Options Trader) is a real-time financial intelligence platform that transforms unstructured social media discourse into quantified, IV-aware options trade recommendations. The system ingests posts from Reddit and 15+ institutional and retail sources, processes them through a 9-stage signal processing DAG, and delivers structured trade ideas with confidence scores, options strategies, and risk annotations.

This document provides a complete mathematical specification of the pipeline for quantitative analysts and ML engineers. It covers the NLP engine (10 passes, custom-built without external NLP libraries), the dual-layer credibility scoring architecture (12-factor heuristic + GradientBoosting over 32 features), the LLM circuit breaker, and the IV-aware options strategy selector. All formulas, feature vectors, and design guarantees are documented as they exist in production.

# 1. Signal Processing DAG — Pipeline Overview

The pipeline is structured as a directed acyclic graph (DAG) in which each stage produces a typed output consumed by the next. Execution is conditional: if a stage fails to produce a valid output, downstream stages do not execute and the signal is stubbed with zero confidence.



# 2. Stage 3 — NLP Engine (10 Modules)

The NLP engine is entirely custom-built. No external libraries (spaCy, NLTK, transformers) are used. It executes 7 sequential passes over raw post text, producing a typed NLPResult struct that feeds Stage 4 event classification.

## 2.1 Pass Architecture Summary

| Pass | Mechanism | Output / Formula |
|---|---|---|
| Pass 1: Lexicon Lookup | 500+ term lexicon, n-gram priority (3→2→1) | `Polarity ∈ [−1,1], Intensity ∈ [0,1]` |
| Pass 2: Negation Flip | Window = 3 tokens after negation keyword | `polarity × −0.8, intensity × 0.7` |
| Pass 3: Intensifier/Diminisher | Lexical boosters & dampeners | `Intensity × 1.4 or × 0.5` |
| Pass 4: ALL-CAPS + Char Repeat | Uppercase detection, repeated char regex | `Boost × 1.3 / min(1.5, 1.0 + 0.1n)` |
| Pass 5: Sarcasm Detection | 8 heuristic rules → sarcasm_prob ∈ [0,1] | `If prob > 0.6: polarity ×= −0.7` |
| Pass 6: Conviction Scoring | High/low conviction phrase ratio | `conviction ∈ [0.10, 0.95]` |
| Pass 7: Weighted Aggregation | Intensity-weighted polarity mean | `NLPResult struct → Stage 4` |

## 2.2 Pass 2: Negation Flip — Formal Specification

Negation keywords: {"not", "no", "never", "without"}. Window size: 3 tokens following the keyword.

```
polarity_new  = −polarity × 0.8      # flip + 20% dampening
intensity_new = intensity × 0.7       # weakened certainty under negation
```

## 2.3 Pass 5: Sarcasm Detection — 8-Rule Heuristic

Sarcasm probability is computed as the clamped sum of rule activations. If the probability exceeds 0.6, the polarity is inverted — this is the primary defence against WSB meme posts generating false bullish signals.

```
Rule activations:
  🤡 emoji present                          → +0.40
  Eyeroll emoji present                     → +0.35
```

```
  ALL-CAPS positive + negative context       → +0.35
  Rhetorical question + positive statement   → +0.35
  Known sarcastic phrase match               → +0.25 to +0.50
  Emoji contradiction                        → +0.30
  Quoted positive words                      → +0.25
  3+ 🚀 rockets AND < 15 words               → +0.15

sarcasm_prob = min(1.0, Σ activations)

if sarcasm_prob > 0.6:
    polarity_new  = −polarity × 0.7
    intensity_new = intensity × 0.7
```

### 2.4 Pass 6: Conviction Scoring

```
raw = (high_conviction_phrases − low_conviction_phrases) / total_phrases  ∈ [−1, 1]
conviction = 0.5 + raw × 0.45   ∈ [0.10, 0.95]

High-conviction lexicon: {"definitely", "absolutely sure", "no doubt", ...}
Low-conviction lexicon:  {"might", "could be", "possibly", "maybe", ...}
```

### 2.5 Pass 7: Weighted Aggregation

```
polarity  = Σ(polarity_i × intensity_i) / Σ(intensity_i)     # intensity-weighted
mean

count_factor = min(1.0, n_signals / 10)                       # saturates at 10
signals
intensity    = mean(intensity_i) × (0.5 + 0.5 × count_factor)

→ NLPResult(polarity, intensity, conviction, sarcasm_prob, bullish_count,
bearish_count)
```

### 2.6 Stance Derivation (Deterministic Rule Tree)

```
if thread_agreement_with_op < 0.3   → "mixed"      # community disagrees
elif sarcasm > 0.7 AND |polarity| < 0.2   → "unknown"
elif polarity > +0.15   → "bullish"
elif polarity < −0.15   → "bearish"
elif bullish_count > 0 AND bearish_count > 0   → "mixed"
else   → "unknown"
```

## 3. Stage 6 — Dual-Layer Credibility Scoring

Stage 6 runs two scoring models sequentially. The heuristic scorer is always executed. The GradientBoosting ML scorer is executed optionally, with its result stored alongside for A/B monitoring and future calibration. The heuristic confidence score feeds downstream stages; the ML P(win) score is stored in metadata.

## 3.1 Layer 1: 12-Factor Heuristic Scorer

Additive adjustment model over a base confidence inherited from upstream stages:

```
final_confidence = clamp(0.05, 1.0,  base_confidence + Σ δ_i)
```

| Factor | δ | Rationale |
|---|---|---|
| Institutional RSS source (FDA/SEC/DoD) | `+0.15` | High-signal, low-noise external feeds |
| DD flair + body length ≥ 200 chars | `+0.15` | Indicates genuine due diligence post |
| Earnings rumor classification | `−0.10` | ~11% historical win rate on rumours |
| Crosspost flag | `−0.10` | Reposts carry no new informational value |
| Entity count ≥ 5 | `−0.08` | Watchlist dump heuristic, not actionable |
| Source: r/WallStreetBets | `−0.05` | Empirically low signal-to-noise |
| Source: r/options or r/thetagang | `+0.05` | Higher-quality options discourse |
| Author karma > 50,000 | `+0.10` | Established, credible contributor |
| Author account age < 30 days | `−0.05` | New account proxy for low trust |
| NLP sarcasm probability > 0 | `−min(0.15, sarcasm × 0.2)` | Proportional sarcasm penalty |
| NLP conviction score > 0.7 | `+0.05` | High-certainty language detected |
| NLP thread contrarian flag | `−0.05` | Community disagrees with OP |

## 3.2 Layer 2: GradientBoosting ML Scorer

The ML model outputs P(win) — the probability that the signal resolves as profitable within the specified horizon. It is trained on historical signals with known outcomes using the same 32-feature extraction function used in live inference, guaranteeing zero train-test feature skew.

### 3.2.1 Feature Vector (32 Dimensions, Fixed Ordinal)

| Idx | Feature | Idx | Feature | Idx | Feature |
|---|---|---|---|---|---|
| `[0]` | post_score | `[11]` | nlp_intensity | `[22]` | pc_ratio |
| `[1]` | num_comments | `[12]` | nlp_conviction | `[23]` | volume_ratio |
| `[2]` | upvote_ratio | `[13]` | nlp_sarcasm | `[24]` | author_karma_log |
| `[3]` | is_crosspost | `[14]` | nlp_urgency | `[25]` | author_age_days |

| Idx | Feature | Idx | Feature | Idx | Feature |
|---|---|---|---|---|---|
| [4] | body_length | [15] | nlp_thread_consensus | [26] | event_type_idx |
| [5] | entity_count | [16] | nlp_thread_agreement | [27] | stance_idx |
| [6] | comment_to_score | [17] | nlp_contrarian | [28] | horizon_idx |
| [7] | trend_score | [18] | nlp_has_data | [29] | is_rss |
| [8] | score_rate (c/s) | [19] | market_cap_log | [30] | subreddit_quality |
| [9] | comment_rate (c/s) | [20] | pct_1d | [31] | (spare/padding) |
| [10] | nlp_polarity | [21] | atm_iv | | |

### 3.2.2 Safe Coercions

```
log_features:    log10(max(x, 1))          # prevents log(0) on karma, market_cap
all_features:    NaN → 0.0,  inf → 0.0     # stable under missing market data
```

### 3.2.3 Inference

```
features = extract_features_from_event(event)     # → list[float], len = 32
probs    = model.predict_proba([features])[0]     # → [P(loss), P(win)]
ml_confidence = clamp(0.05, 0.95, probs[1])      # P(win), hard-bounded

meta["ml_credibility"] = {
    "ml_confidence":     ml_confidence,
    "heuristic_confidence": heuristic_score,
    "model_path":        str
}
```

### 3.2.4 Training Protocol

Training is executed via train.py. The pipeline pulls historical signals with resolved outcomes from the production database, extracts the identical 32-element feature vector via extract_features_from_row(), labels each signal by win/loss, and fits a GradientBoostingClassifier. The serialised model hot-reloads on the next pipeline run — no service restart required.

Key guarantee: extract_features_from_event() (live path) and extract_features_from_row() (training path) are the same function applied to different data representations. This structural guarantee eliminates train-test skew as a failure mode.

# 4. Stage 7 — LLM Reasoning & Circuit Breaker

An LLM is invoked to produce a thesis, refine stance, and identify risk factors. Because LLM API calls are the highest-latency and highest-failure-rate component in the pipeline, a circuit breaker is implemented with a hard 3-failure cutoff.

```
State: consecutive_failures ∈ {0, 1, 2, 3}

if consecutive_failures ≥ 3:
    return stub_packet()                    # hard cutoff — no retry attempted

try:
    packet = llm_reason(event)
    consecutive_failures = 0               # reset on success
    return packet
except:
    consecutive_failures += 1
    if consecutive_failures ≥ 3:
        log("LLM CIRCUIT BREAKER OPEN")
    return stub_packet()
```

When the LLM succeeds, its confidence and stance values override the heuristic confidence from Stage 6. This LLM-calibrated confidence then propagates into Stage 8 trade construction and the final quality score.

# 5. Stage 8 — IV-Aware Trade Construction

Stage 8 selects an options strategy conditioned on both the stance from Stage 7 and the implied volatility regime from Stage 5. The core financial insight: volatility regime determines whether premium should be sold or purchased.

## 5.1 Volatility Regime Gate

```
high_iv = (atm_iv > 0.50)

High IV regime → option prices inflated → selling premium has higher expected value
              → credit spreads, iron condors

Low IV regime  → option prices cheap    → buying premium has higher expected value
              → debit spreads, straddles
```

## 5.2 Strategy Selection Matrix

| Stance | High IV (ATM IV > 0.50) | Low / Normal IV (ATM IV ≤ 0.50) |
|---|---|---|
| Bullish | Bull Put Credit Spread | Bull Call Debit Spread |

| Stance | High IV (ATM IV > 0.50) | Low / Normal IV (ATM IV ≤ 0.50) |
|---|---|---|
| Bearish | Bear Call Credit Spread | Bear Put Debit Spread |
| Mixed / Unknown | Iron Condor | Long Straddle |

### 5.3 Max Loss Formulas

```
Debit / Credit Spread:   max_loss = |strike_long − strike_short| × 100
Long Straddle:           max_loss = price × 0.03 × 100 × 2    # 3% premium estimate
per side
Iron Condor:             max_loss = |inner_wing_width| × 100
```

### 5.4 Quality Score (Composite)

```
score =   confidence × 0.7
        + 0.10  if event_type != "other"
        + 0.10  if len(thesis) > 50 chars
        + 0.10  if recommended_structures from LLM non-empty
        − 0.10  if len(risk_notes) > 3
→ clamp(0.0, 1.0)
```

## 6. Stage 6.5 — Adaptive Suppression (Feedback Gate)

Stage 6.5 implements a closed-loop learning mechanism. After sufficient historical data accumulates, signals that have demonstrated consistent poor performance are suppressed before consuming LLM compute in Stage 7.

```
if signal_historically_weak(event):
    event.confidence = 0.0
    return stub_signal()          # skip Stages 7 and 8 entirely
```

This gate is the primary mechanism by which ROT's win rate improves over time without retraining the ML model. Historical weakness is determined by the feedback loop that writes resolved outcomes back to the database after each market session.

## 7. Failure Mode Analysis & Design Guarantees

The following table documents the primary failure modes considered during system design, the architectural mitigation for each, and the engineering rationale.

| Failure Mode | Mitigation | Engineering Rationale |
|---|---|---|
| Sarcastic/meme posts | 8-rule sarcasm detector with | Prevents WSB noise from entering |

| Failure Mode | Mitigation | Engineering Rationale |
|---|---|---|
| generate false bullish signals | polarity inversion | signal stream |
| Single scoring model failure | Dual-layer: heuristic always on, ML optional | Zero single-point-of-failure in confidence scoring |
| LLM API instability | Circuit breaker — 3 consecutive failure hard cutoff | Pipeline never hangs; stub packet issued automatically |
| Incorrect options strategy in wrong volatility regime | ATM IV regime gate at 0.50 threshold | Premium selling/buying correctly positioned to IV |
| Historical losers continue firing | Stage 6.5 adaptive suppression from feedback loop | Self-improving win rate via closed-loop learning |
| Train-test feature skew | Identical 32-feature extraction for live and training | ML model generalises to production without leakage |
| NaN propagation into GradientBoosting | _safe_float() and _safe_log10() coercions on all features | Numerically stable inference under any market condition |
| Confidence degeneracy (zero values) | Confidence clamped to [0.05, 1.0] from below | Prevents zero-weight signals from bypassing downstream filters |

# 8. System Properties & Production Metrics

## 8.1 Codebase Statistics

- 165,000+ lines of production Rust
- 335 files, 952 tests
- Test-to-production ratio: 1.57:1
- CodeQL security scan: 0 open findings (425 resolved)
- SQLite schema: 33 tables

## 8.2 Data Sources

- Reddit (r/wallstreetbets, r/options, r/thetagang, r/investing, and others)
- Institutional RSS feeds: FDA, SEC, DoD, Federal Reserve
- Financial data APIs: price feeds, options chains, Greeks, IV surface
- News aggregators: Financial Times, Seeking Alpha, MarketWatch, Investing.com
- Total integrated sources: 15+

### 8.3 Week 1 Validated Signals (Live Production)

| Ticker | Signal | Move | Notes |
|---|---|---|---|
| MASI | $9.9B Danaher acquisition — detected from FT leak, Sunday night | +34% | Caught ~12 hours pre-announcement |
| ZIM | $4.2B Hapag-Lloyd buyout — flagged over Presidents Day weekend | +25% | 1 full trading day lead time |
| OLB | PayPal partnership announcement — detected same day | +137% | Product news classification, 50% confidence |
| CMPS | WSB hype surge — read 3 days before Phase 3 results | +39% | earnings_rumor classification |

# 9. MCP Server Integration

ROT exposes its signal pipeline as a Model Context Protocol (MCP) server, enabling direct integration into LLM clients including Claude. This makes ROT the first financial intelligence platform accessible as a native LLM tool rather than a standalone dashboard.

### 9.1 Exposed Tools

- get_trending_tickers(hours, limit) — Returns tickers ranked by signal volume with confidence scores
- get_signals(ticker, stance, limit) — Returns latest trading signals with AI-generated analysis
- get_sentiment(ticker) — Bull/bear/mixed ratio, net sentiment score [−1.0, +1.0]
- get_market_overview() — 30-day aggregate: win rate, signal distribution, average confidence
- get_unusual_activity(hours, limit) — IV spikes, volume surges, open interest anomalies
- get_sports_feed(league, limit) — Sports betting intelligence with line mover scores
- search_signals(query, limit) — Full-text search across all signal metadata

# 10. Deployment Architecture

- Runtime: Railway.app (production), Rust async runtime (Tokio)
- Storage: SQLite with WAL mode (33 tables), append-only signal log
- Delivery: WebSocket (real-time), Discord webhook, email digest

- API: FastAPI (Python) for dashboard and MCP server
- ML model: Serialised GradientBoostingClassifier, hot-reload on pipeline restart
- Security: CodeQL A-grade, 425 findings resolved, zero open
- Availability: Active in 29 countries as of February 2026

# Appendix A — Confidence Clamping Justification

All confidence values in the pipeline are clamped to [0.05, 1.0]. The lower bound of 0.05 is intentional: a confidence of exactly 0.0 would cause downstream filtering logic to treat the signal as non-existent rather than low-confidence. The 5% floor preserves the signal in storage and delivery while correctly deprioritising it in quality-ranked output views.

Earnings rumour signals are additionally hard-capped at 0.25 regardless of heuristic or ML output. This cap is derived from empirical historical win rate data (~11%) and prevents the ML model from overriding a known low-precision signal class during early training phases when data volume is insufficient to suppress it naturally.

# Appendix B — Sarcasm Detection Rationale

The 8-rule sarcasm heuristic exists because WSB (r/WallStreetBets) post language is structurally unlike standard English sentiment. Phrases like "this is the play" or posts with multiple rocket emojis are frequently ironic. Standard polarity lexicons trained on financial news corpora systematically misclassify these as bullish signals.

The double-defence architecture (sarcasm penalty in Layer 1 heuristic scoring AND polarity inversion in the NLP pass) ensures that even if the NLP pass underestimates sarcasm probability, the credibility scorer applies a proportional penalty. Empirical testing on WSB data showed that without this layer, false bullish signal rate on meme posts exceeded 60%.

# Appendix C — Platform & Contact

Live platform: web-production-71423.up.railway.app

GitHub: github.com/Mattbusel/Reddit-Options-Trader-ROT

Author: Matthew Charles Busel | AI Engineer & Tech Writer

LinkedIn: linkedin.com/in/matthewbusel